Unit – 3

Object-Oriented Software Design and Development

Interaction Diagrams • Sequence • Collaboration • Class Diagrams • State Chart • Reuse Concepts



MCA – 212 • Object-Oriented Software Engineering

Dr Madhavi Dave

Learning Objectives

What students should be able to do after this module

01 • Interaction Diagrams

- Identify objects, messages and lifelines that realize a use case.

02 • Sequence Diagrams

- Model time-ordered message flow between objects, including alt/opt/loop fragments.

03 • Collaboration Diagrams

- Model the same interaction as a structural, numbered-message diagram.

04 • Class Diagrams

- Design classes, attributes, operations and relationships with correct multiplicity.

05 • State Chart Diagrams

- Model an object's lifecycle as states, events, guards and transitions.

06 • Reuse Concepts

- Apply inheritance, composition, patterns and components to reuse designs safely.

- Apply every diagram to the same two running case studies: an ATM and a Library Management System.

SECTION 1

Interaction Diagrams

How objects talk to each other while a use case executes

What Is an Interaction Diagram?

The dynamic view of an OO design

- An interaction diagram shows how a group of objects collaborate — the messages they send each other to carry out one use case or scenario.
- It turns a use case's plain-text flow ("student logs in, system validates, …") into a precise, checkable picture of object communication.
- UML defines two equivalent notations for the same interaction: the Sequence Diagram (time-focused) and the Collaboration / Communication Diagram (structure-focused).

Object / Lifeline

- An instance participating in the interaction; its dashed vertical line shows its lifetime.

Message

- A request from one object to another — a method call, shown as a labelled arrow.

Activation

- A thin bar showing the period an object is actively executing a method.

Two Views of the Same Interaction

Same information, two complementary layouts

Aspect	Sequence Diagram	Collaboration Diagram
Emphasis	Time ordering of messages	Structural links between objects
Layout	Objects across top, time flows down	Objects free-form, links radiate out
Message order shown by	Vertical position	Sequence numbers on each link
Best for	Understanding timing / long scenarios	Understanding object relationships

When to use interaction diagrams

Use them to verify a use case is realizable, to design/debug object collaborations, and to hand a precise message contract to developers before coding.

SECTION 2

Sequence Diagrams

Modeling object interaction as it unfolds over time

Sequence Diagram — Notation

Reading the diagram: objects across the top, time flowing down

Lifeline

- A dashed vertical line dropping from an object box; represents the object's existence over time.

Activation Bar

- A thin rectangle on the lifeline marking when that object is actively processing.

Synchronous Message

- Solid line, filled arrowhead — caller waits for the callee to finish.

Asynchronous Message

- Solid line, open/thin arrowhead — caller continues without waiting.

Return Message

- Dashed line, open arrowhead — the reply flowing back to the caller.

Create / Destroy

- A message to «create» a new object, or an X marking «destroy» at end of life.

Combined Fragments

Modeling conditions and loops inside a sequence

Fragment	Meaning	Example
alt	Alternative — one of several branches executes (if/else)	alt [PIN valid] / [PIN invalid]
opt	Optional — the enclosed messages run only if the guard holds	opt [balance < threshold]
loop	Repetition — the enclosed messages repeat while a condition holds	loop [for each item in cart]
par	Parallel — enclosed operands execute concurrently	par [send email] / [update log]

Notation

Drawn as a labelled rectangle wrapped around the relevant messages, with the keyword in the top-left corner and the guard condition in square brackets.

Case Study — ATM Cash Withdrawal

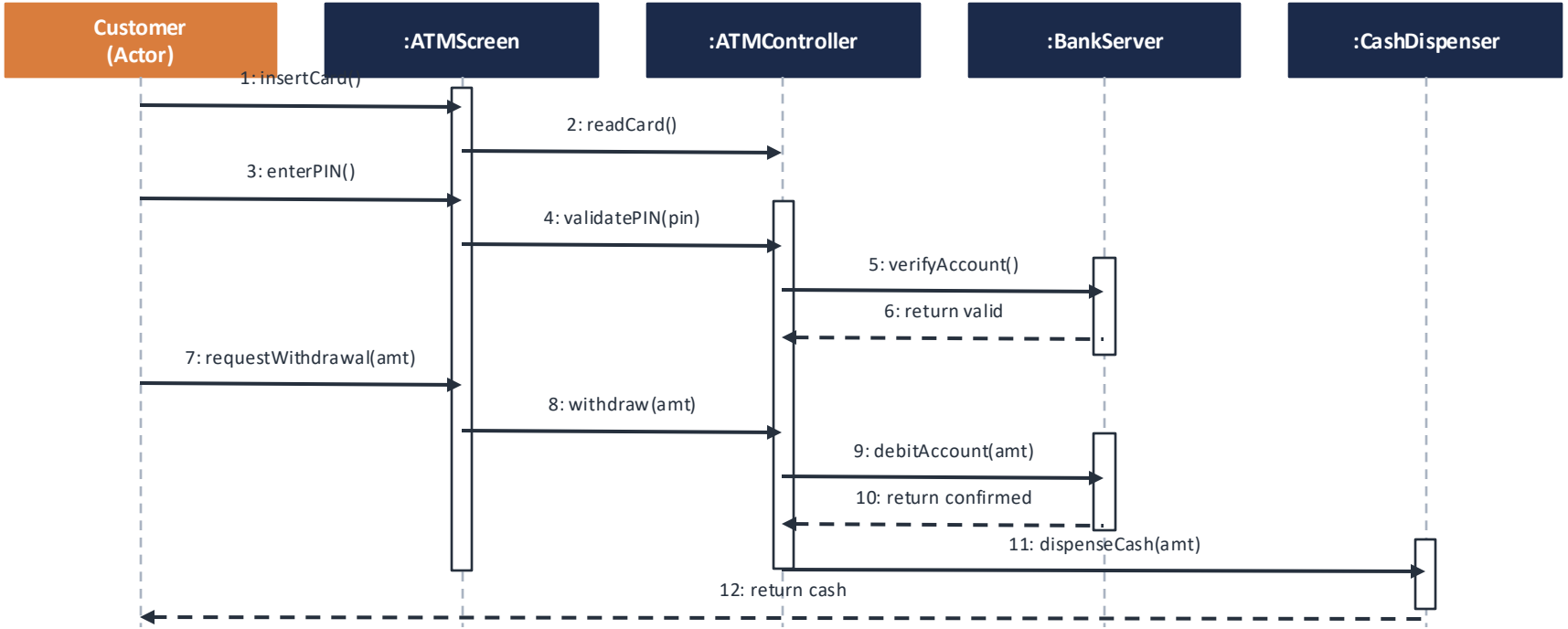
Setting up the scenario we will diagram

- Scenario: a customer inserts a card, enters a PIN, requests a withdrawal, and receives cash.
- Participating objects: Customer (actor), :ATMScreen, :ATMController, :BankServer, :CashDispenser.

Step	Message	From → To
1	insertCard()	Customer → ATMScreen
2	readCard() / enterPIN() / validatePIN(pin)	ATMScreen ↔ ATMController
3	verifyAccount()	ATMController → BankServer
4	requestWithdrawal(amt) / debitAccount(amt)	Customer / ATMController → BankServer
5	dispenseCash(amt)	ATMController → CashDispenser

Sequence Diagram — ATM Cash Withdrawal

Time flows downward; each column is one object's lifeline



Reading the Sequence Diagram

Step-by-step walkthrough & common mistakes

Walkthrough

Messages 1–4: the card and PIN are captured and validated with the controller.

Messages 5–6: the controller confirms the account with the bank (synchronous call + dashed return).

Messages 7–10: the withdrawal amount is requested and the account is debited.

Messages 11–12: cash is physically dispensed back to the customer.

Common Mistakes

Forgetting the dashed return message — every synchronous call should show its reply.

Missing activation bars, making it unclear who is "busy" at each point.

Skipping the actor lifeline, so the trigger for the whole scenario is unclear.

Drawing messages between lifelines that never actually interact.

SECTION 3

Collaboration (Communication) Diagrams

The same interaction, viewed through the object structure

Collaboration Diagram — Notation & Purpose

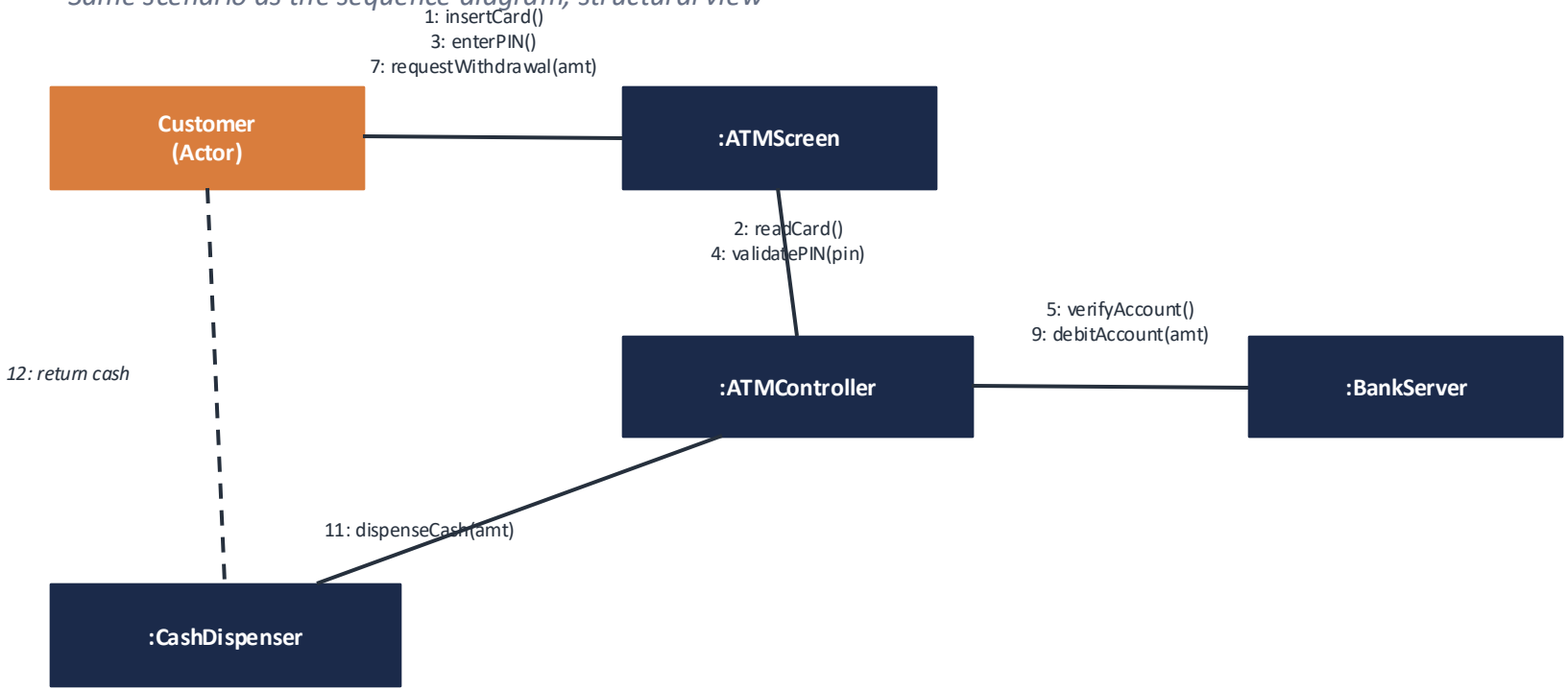
Objects, links and numbered messages

- Objects are drawn as simple boxes wherever they fit best on the page — layout reflects their structural relationships, not time.
- A link (plain line) is drawn between two objects that exchange at least one message.
- Every message is labelled with a sequence number (1, 2, 3 …) so the reader can reconstruct the order, even though position doesn't show time.
- Nested numbering (1.1, 1.2 …) can show messages that happen inside another message's processing.

Aspect	Sequence	Collaboration
Time order shown by	Vertical position (implicit)	Sequence numbers (explicit)
Object relationships shown by	Not shown directly	Link layout — explicit

Collaboration Diagram — ATM Cash Withdrawal

Same scenario as the sequence diagram, structural view



Same 12 messages as the sequence diagram — but now grouped by which objects talk to which.

Reading the Diagram & When to Prefer It

Collaboration diagrams shine when structure matters most

Reading it

Follow the numbers 1→12 around the diagram, regardless of position on the page.

The link layout itself tells you ATMController is the hub — it talks to every other object.

BankServer and CashDispenser never talk to each other directly — no link is drawn between them.

Prefer Collaboration When...

You want to highlight which objects are structurally coupled (useful for impact analysis).

The interaction is short, so time-ordering is easy to track via numbers alone.

You are documenting object relationships alongside a class diagram.

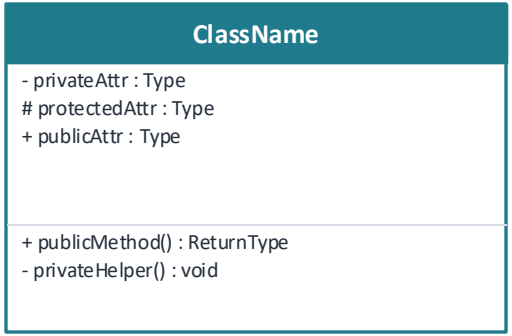
SECTION 4

Class Diagrams

The static blueprint: classes, attributes, operations and relationships

Class Diagram — Notation

The three-compartment class box



← Class name (bold, centered)

← Attributes (with visibility & type)

← Operations / methods

Symbol	Visibility
+	public
-	private
#	protected

Relationship Types & Multiplicity

How classes connect to one another

Relationship	Meaning	Notation
Association	A general "uses / knows about" link between peer classes	plain line
Aggregation	"Has-a", whole-part; the part can exist independently	line + open ◇ diamond at whole
Composition	Strong "has-a"; the part's lifetime depends on the whole	line + filled ◆ diamond at whole
Generalization	"Is-a" — inheritance between super/subclass	line + hollow ▷ triangle at parent
Dependency	One class relies on another only temporarily (e.g. a parameter)	dashed line + open arrow

Multiplicity is written near each end of a line: 1, 0..1, 0..*, 1..*, or an exact range like 3..5.

Case Study — Library Management System

Setting up the scenario we will diagram

- A Library aggregates many Books. Members register with the Library and take out Loans.
- Every Loan is for exactly one Book and belongs to exactly one Member.
- Student and Faculty are two specialized kinds of Member (inheritance).

Library

- - libraryId, - name
- + addBook(), + registerMember()

Book

- - isbn, - title, - copiesAvailable
- + checkAvailability()

Member

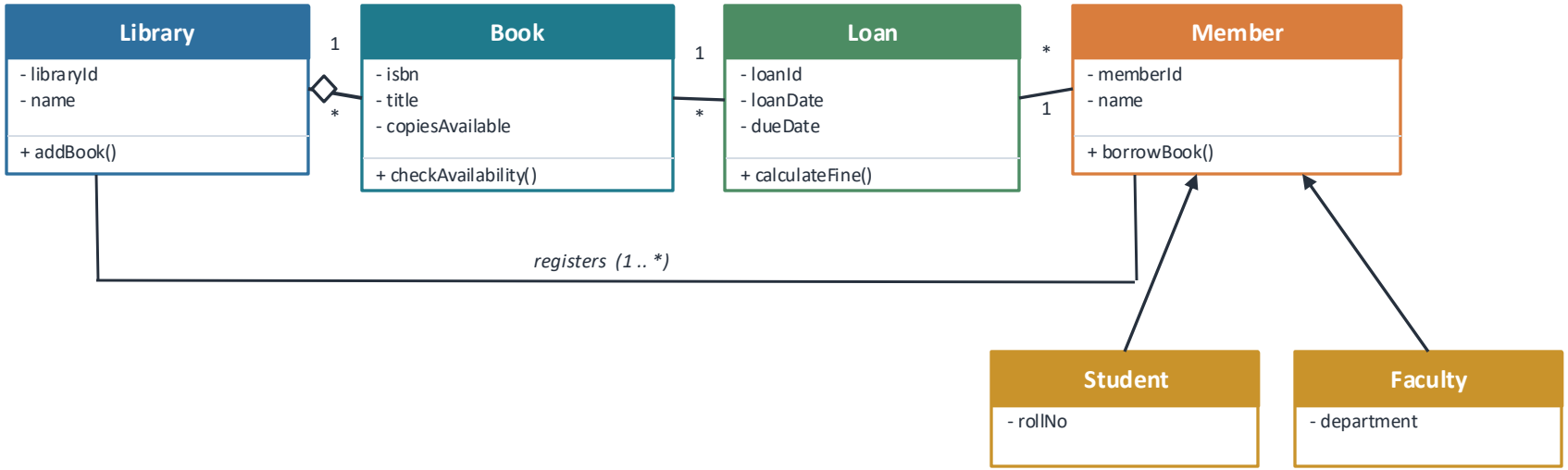
- - memberId, - name, - email
- + borrowBook(), + returnBook()

Loan

- - loanId, - loanDate, - dueDate
- + calculateFine()

Class Diagram — Library Management System

Association, aggregation and generalization together



Reading the Class Diagram

Relationships explained

- **Library ◇— Book (aggregation, 1 .. *)**
- A Library holds many Books, but a Book can in principle be catalogued before it belongs to a specific library instance — the part can outlive the whole.
- **Book — Loan — Member (association chain)**
- Every Loan links exactly one Book to exactly one Member; a Member can have many Loans over time.
- **Library — Member (association, "registers")**
- A Library registers many Members; the relationship is a simple peer link, not ownership.
- **Student ▷ Member, Faculty ▷ Member (generalization)**
- Both subclasses inherit memberId, name and borrowBook()/returnBook(), adding only what's specific to them (rollNo, department).

SECTION 5

State Chart Diagrams

Modeling one object's lifecycle across events

State Chart — Notation

Building blocks of a state machine

Initial State

- A filled black circle — the default starting point of the object's life.

State

- A rounded rectangle naming a condition the object is in (e.g. Confirmed).

Transition

- An arrow from one state to another, labelled event [guard] / action.

Guard Condition

- A boolean check in square brackets that must hold for the transition to fire.

Action

- An activity carried out as the transition fires, written after a slash.

Final State

- A circle with a filled inner dot — the object's life/process has ended.

Case Study — Order Lifecycle

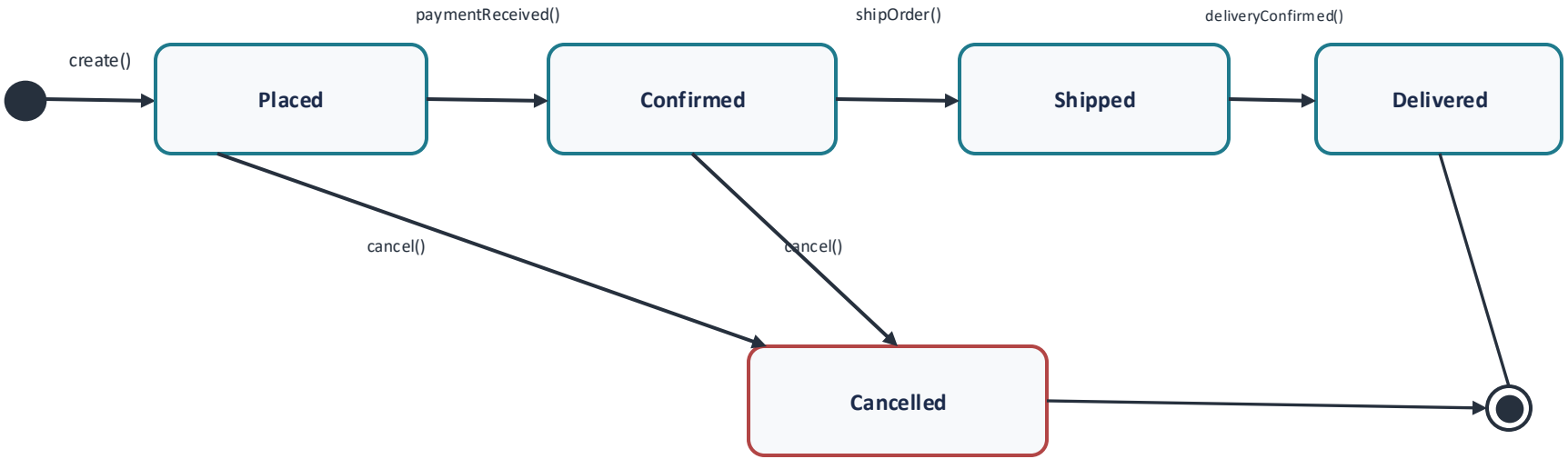
Setting up the scenario we will diagram

- Object: a single Order in an e-commerce system, from the moment it is placed to its final outcome.
- It moves through Placed, Confirmed, Shipped and Delivered — or is Cancelled along the way.

Transition	Trigger / Guard	Action
(start) → Placed	create()	—
Placed → Confirmed	paymentReceived()	send confirmation email
Placed / Confirmed → Cancelled	cancel()	refund if paid
Confirmed → Shipped	shipOrder()	generate tracking ID
Shipped → Delivered	deliveryConfirmed()	close order

State Chart Diagram — Order Lifecycle

From creation to delivery or cancellation



Guards can gate a transition further, e.g. cancel() [not yet shipped] — written in square brackets after the event.

Reading the Diagram & Composite States

Nesting states for complex objects

Walkthrough

The order starts its life the moment create() fires, entering Placed.

paymentReceived() advances it to Confirmed; from either Placed or Confirmed, cancel() diverts it to Cancelled.

Once Shipped, the only forward path is Delivered — both Delivered and Cancelled lead to the final state.

Composite (Nested) States

A state can contain its own mini state-machine — e.g. Shipped could nest InTransit → OutForDelivery.

This keeps the top-level diagram simple while still capturing detail where it matters.

Composite states share transitions: cancel() from the outer Shipped state applies to every nested sub-state.

SECTION 6

Reuse Concepts

Designing so tomorrow's project doesn't start from zero

Why Software Reuse?

Building on what already works

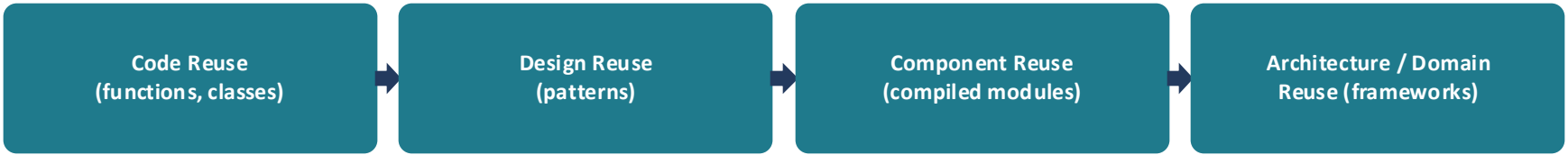
- Reuse means using existing software assets — code, designs, components, or whole architectures — in a new context instead of building from scratch.
- It directly reduces development time and cost, since proven logic doesn't need to be re-written or re-tested from zero.
- Reused components are typically more reliable — they have already been exercised in production by earlier projects.
- Object orientation was designed with reuse in mind: encapsulation, inheritance and polymorphism all exist partly to make reuse practical.

The economics of reuse

Every hour spent designing a component for reuse is repaid many times over once a second, third, or tenth project reuses it without touching the source.

Levels of Reuse

Reuse gets more valuable — and harder — the higher it goes



Code Reuse

- Copying or importing individual functions, classes or libraries into new code.

Design Reuse

- Reapplying a proven design pattern or algorithm structure, rewritten for the new context.

Component Reuse

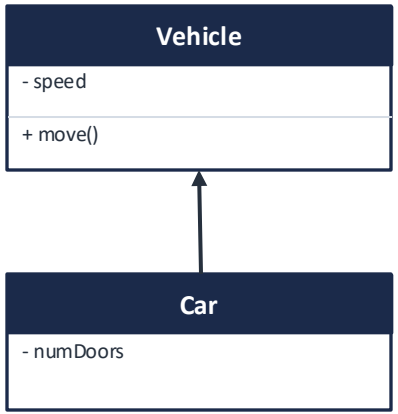
- Plugging in a pre-built, pre-tested module (e.g. a payment SDK) through a defined interface.

Architecture / Domain Reuse

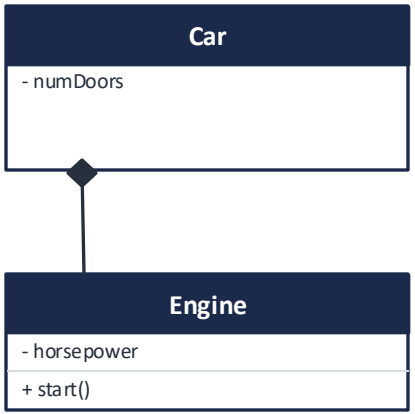
- Adopting an entire application framework or reference architecture for a whole product family.

White-Box vs Black-Box Reuse

Inheritance vs. Composition as reuse mechanisms



White-box reuse: Car inherits Vehicle's implementation directly ("is-a")



Black-box reuse: Car simply holds a reference to Engine ("has-a")

Rule of thumb: prefer composition (black-box) over inheritance (white-box) — it keeps classes loosely coupled and easier to change independently.

Design Patterns as Reuse

Reusing proven solutions, not just code

- A design pattern is a reusable, named solution to a recurring design problem — reuse of expertise rather than a specific line of code.
- The Gang of Four (GoF) catalog groups patterns into three families:

Creational

- Control how objects are created.
- e.g. Singleton, Factory Method

Structural

- Compose classes/objects into larger structures.
- e.g. Adapter, Decorator

Behavioral

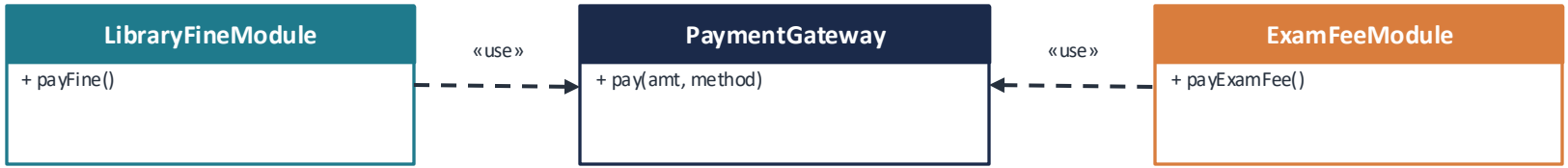
- Manage communication & responsibility between objects.
- e.g. Observer, Strategy

Example: an Observer pattern lets the Library's Book list notify a WaitlistNotifier whenever a copy becomes available — without either class needing to know the other's internals.

Case Study — Reusable PaymentGateway

One component, two unrelated modules

- A university builds a single PaymentGateway component with a stable interface: pay(amount, method).
- It is reused, unmodified, by two completely different modules of the same OO system.



Why this works

Both modules depend only on PaymentGateway's public interface (dependency, not inheritance) — the gateway can be upgraded internally without either module changing.

Benefits & Risks of Reuse

Reuse is powerful, but not free

Benefits

Faster development — less code written and tested from scratch.

Higher reliability — reused assets have already been battle-tested.

Lower cost across a portfolio of related projects.

Consistency — shared components enforce a common look, behavior and quality bar.

Risks

Tight coupling to a reused component can make future upgrades harder if its interface changes.

A defect in a widely-reused component propagates to every consumer.

Licensing, versioning and long-term support obligations for third-party assets.

Reuse-for-its-own-sake can force an awkward fit and add unnecessary complexity.

How the Diagrams Fit Together

One design story, seen from three angles



- A use case's flow is first modeled as a Sequence or Collaboration diagram — who talks to whom, and in what order.
- The objects that appear become classes in the Class Diagram, with attributes, operations and relationships fixed.
- Any class with a rich lifecycle (Order, Loan, Book) gets its own State Chart to pin down valid transitions.
- Wherever a class or subsystem is generic enough, it is packaged for reuse across future projects.

Comparing the Diagram Types

Pick the right diagram for the right question

Diagram	Shows	Question it answers
Sequence	Time-ordered messages between objects	What happens, in what order?
Collaboration	Structural links + numbered messages	Which objects are coupled?
Class	Classes, attributes, operations, relationships	What does the system look like statically?
State Chart	One object's states, events and transitions	What states can this object be in?

Key Takeaways

From dynamic behavior to a reusable static design

- Interaction diagrams (Sequence & Collaboration) capture the same behavior from two complementary angles — use whichever communicates best to your audience.
- Class diagrams are the stable backbone of the design; get relationships and multiplicity right before writing code.
- State charts are worth drawing for any class whose objects have a meaningful lifecycle with rules about what can happen next.
- Design for reuse deliberately — favor composition over inheritance, and expose components through small, stable interfaces.